ChatGPT

BIM Photo Sync 통합 구축 기획보고서

경영 요약

BIM Photo Sync는 현장 사진을 단순 저장 자산이 아니라 BIM 공간 정보와 연결된 운영 데이터로 전환하는 시스템이다. 최종 제품은 웹 관리자 콘솔, 모바일 앱, Revit Add-in, AI 분석 워커, 보고서 생성기, 선택형 APS 기반 웹 모델 뷰어, 그리고 자연어 자동화를 위한 MCP 계층으로 구성된다. 웹에서의 모델 조회는 Autodesk [1] 의 Autodesk Platform Services Viewer SDK와 Model Derivative API를 사용하고, Revit 내부의 선택 이벤트·공정 색상 표시·Room 동기화는 Revit API 기반 Add-in으로 분리하는 것이 가장 안정적이다. Autodesk 공식 문서에 따르면 Viewer SDK는 웹사이트에서 2D·3D 설계 모델을 표시하고 상호작용하기 위한 JavaScript 라이브러리이며, Model Derivative API는 설계 파일을 SVF/SVF2로 변환하고 객체 계층, 속성, 지오메트리, 썸네일을 추출한다. Revit API 개발은 Revit 2025 기준 .NET 8.0, `RevitAPI.dll`, `RevitAPIUI.dll`을 전제로 한다. [2]

이 문서는 "사진 업로드 앱"을 만드는 문서가 아니다. Room을 기준으로 사진, 공정 상태, AI 분석, 보고서, Revit 연동, 웹 뷰어, 관리자 워크플로, Agent 자동화를 모두 하나의 데이터 모델 위에서 설계하는 회사 제출용 통합 사양서다. 또한 본 보고서는 기존 구현의 코드나 구조를 재사용하지 않는 **신규 구축안**을 전제로 하며, 기존 시스템은 데이터 이관 대상일 뿐 애플리케이션 기반 자산으로 보지 않는다. 이 전제는 현재 대화에서 정리된 요구사항, 즉 웹·앱 동시 사용, 관리자 모바일 사용, Revit Room 중심 데이터 관리, Add-in과 APS의 역할 분리, MCP 기반 Agent 자동화, AI 검토-승인 워크플로를 통합한 결과다.

본 기획안의 핵심 결론은 다음과 같다. 첫째, 시스템의 기준 객체는 사진이 아니라 Room이다. 둘째, Revit은 서비스 DB가 아니라 BIM 작업 도구다. 셋째, AI는 분석 보조자이며 최종 판단자는 관리자다. 넷째, 웹과 앱은 같은 API와 같은 DB를 사용해야 한다. 다섯째, MCP는 제품 UI를 대신하는 층이 아니라 자연어 기반 운영 자동화 계층으로 설계해야 한다. Revit 쪽 저장 방식은 shared parameter 또는 extensible storage가 가능하지만, Autodesk 문서가 설명하는 shared parameter의 숨김 처리와 외부 정의 방식을 활용하면 운영·디버깅·수동 검증이 쉬우므로 본 문서는 `BIM_PHOTO_ROOM_ID` shared parameter를 표준으로 채택한다. [3]

범위 요약

구분	포함 범위	제외 범위
사용자 제품	웹 관리자 콘솔, 모바일 앱, Revit Add-in	기존 코드 유지보수, 단기 프로토타입 수선
BIM 연동	Room 동기화, Room 클릭 사진 조회, 공정 상태 색상 오버레이	웹에서의 완전한 Revit 편집
AI	사진 분석, 비교 분석, 보고서 초안, 관리자 검토 워크플로	무검토 자동 확정, 완전 자율 의사결정
문서	PDF, DOCX 우선, HWP 확장	초기 단계의 고급 양식 편집기
자동화	MCP 기반 읽기·검색·보고서·오버레이 자동화	사람 승인 없는 위험 작업 자동 실행

핵심 가정

항목	가정
구축 방식	신규 설계·신규 개발, 기존 코드 재사용 없음
테넌시	회사 단위 데이터 완전 분리
권한	Worker, Manager, Project Admin, BIM Manager, Company Admin, Viewer
Room 기준	Room은 Revit과 백엔드 양쪽에 존재하며 백엔드가 기준 ID를 발급
파일 저장	객체 스토리지 직접 업로드 + 메타데이터 커밋 방식
AI	구조화된 JSON 결과 저장, 관리자 승인 가능
Revit 버전	1차 지원 기준 Revit 2025+
웹 모델	APS Viewer를 관리자/BIM 화면에 우선 적용
모바일 관리자	관리자도 앱 사용 가능하되, 정밀 설정/매핑은 웹 전용
MCP	읽기 작업은 자동 가능, 쓰기 작업은 human-in-the-loop 승인 필요

범위와 가정

이 시스템의 문제 정의는 명확하다. 현장 사진은 지금까지 프로젝트명, 방 이름, 공종, 날짜 같은 텍스트 조합으로만 정리되기 쉬웠고, 그 결과 도면과 사진이 분리되고, Room 이름이 바뀌면 연결이 깨지고, 보고서는 사람 손으로 다시 정리해야 했다. BIM Photo Sync는 이 문제를 “사진 저장”이 아니라 “Room에 귀속된 시공 이력 관리” 문제로 재정의한다. 따라서 사진 업로드, AI 분석, 관리자 승인, 보고서 생성, Revit 조회, 웹 Viewer, 감사 로그까지 모두 프로젝트-층-Level-Room 계층에 종속되도록 설계한다.

사용자와 역할

역할	주된 목표	주요 기능
Worker	현장 사진을 빠르고 정확하게 올린다	촬영, Room 선택, 공종/공사면 입력, 업로드, 내 업로드 조회
Manager	사진과 AI 결과를 검토하고 현장 상태를 관리한다	AI 승인/수정, Room 상태 변경, 빠른 보고서 생성
Project Admin	프로젝트 운영과 보고서를 총괄한다	권한 관리, 보고서 생성, 기준값 관리
BIM Manager	Revit과 데이터 모델을 연결한다	Room sync, Add-in 연결, 색상 오버레이, 충돌 해결
Company Admin	회사 단위 보안과 운영 기준을 관리한다	회사 사용자·프로젝트·정책 관리
Viewer	읽기 전용	현황 조회, 보고서 열람

설계 원칙

원칙	의미
Room 중심	사진이 Room에 붙고, Room이 보고서와 상태의 기준이 된다
Backend 우선	서비스 데이터의 진실 원천은 DB다
AI 보조	AI 결과는 초안이며 관리자 승인 전에는 확정 상태가 아님
Web/App 동기화	웹과 앱은 같은 API·DB·Storage를 사용한다
Add-in 분리	Revit Desktop 내부 조작은 Add-in이 담당한다
Viewer 분리	웹 모델 조회는 APS가 담당한다
MCP 보조	MCP는 업무 자동화 계층이며 핵심 비즈니스 데이터 저장소가 아니다

옵션 비교와 권고안

APS Viewer는 JavaScript 기반 웹 뷰어이고, Model Derivative는 SVF/SVF2 변환과 메타데이터 추출을 담당한다. 반면 이미지 맵은 구현은 빠르지만 BIM 객체 수준의 의미를 제공하지 못한다. 따라서 관리자/BIM 기능에서는 APS Viewer를 쓰고, 모바일이나 초기 화면에서는 단순 도면 이미지 또는 Room 리스트 기반 탐색을 보조적으로 병행하는 하이브리드 접근이 가장 현실적이다. [4]

옵션	장점	단점	복잡도	초기 비용 체감	최종 권고
APS Viewer 단독	BIM 문맥 유지, 2D/3D, 속성 탐색, 확장성 우수	인증·변환·커스터마이징 필요	높음	중간	관리자/BIM 화면에 적합
이미지 맵 단독	빠른 구현, 모바일 가벼움	BIM 의미 없음, 재매핑 비용 큼	낮음	낮음	단독 채택 비권장
하이브리드	초기 속도와 장기 확장성 균형	화면별 설계 분리 필요	중간	중간	최종 권고

객체 스토리지는 S3 presigned URL과 R2 direct upload 모두 가능하다. 공식 문서 기준으로 Amazon Web Services [5] Amazon S3의 presigned URL은 PUT/GET/HEAD와 만료 시간, 체크섬 검증을 지원하고, Cloudflare [6] Cloudflare R2는 S3-compatible API, strong consistency, no egress fees를 제공한다. 따라서 인터페이스는 S3-compatible로 통일하고, 실제 배포는 비용 구조와 조직 선호에 따라 S3 또는 R2를 선택하는 것이 맞다. [7]

옵션	장점	단점	복잡도	비용 민감도	권고
S3	성숙한 생태계, 표준적 권한·보안 도구	대규모 외부 다운로드 시 비용 민감	낮음~중간	중간~높음	엔터프라이즈 기본안
R2	no egress, S3 호환, 직접 업로드 유리	운영팀 경험이 없으면 초기 학습 필요	중간	낮음	미디어 트래픽이 많으면 우선 고려

옵션	장점	단점	복잡도	비용 민감도	권고
추상화 계층	향후 공급자 교체 쉬움	초기 래퍼 설계 필요	중간	낮음	강력 권고

모바일 프런트는 React Native와 Flutter가 모두 가능하다. React Native 공식 문서는 React 경험이 있는 개발자가 native app parity를 얻기 쉽고, 새 앱에는 프레임워크 사용을 권장한다고 설명한다. Flutter 공식 문서는 단일 코드베이스로 모바일·웹·데스크톱까지 배포 가능한 멀티플랫폼 프레임워크라고 설명한다. 본 프로젝트는 웹 관리자 콘솔과 APS Viewer 연계가 핵심이므로 **Next.js 웹 + React Native 모바일**이 기본 권고안이고, 앱이 제품의 절대 중심이고 조직이 Dart 역량을 갖고 있다면 Flutter가 대안이다. [8]

옵션	장점	단점	복잡도	조직 적합성	권고
Next.js + React Native	TypeScript 공유, 웹 관리자 강점, APS 연계 자연스러움	완전한 단일 코드베이스는 아님	중간	높음	최종 권고
Flutter	모바일·웹·데스크톱 단일 코드베이스	APS 웹 연계는 상대적으로 별도 설계 필요	중간~높음	중간	대안
PWA 단독	배포 간단	카메라·오프라인·큐 관리 한계	낮음	낮음	단독 비권장

아키텍처와 기술 선택

본 시스템은 요청 처리 계층, 상태 저장 계층, 비동기 처리 계층, BIM 연동 계층으로 분리한다. API 서버는 stateless하게 유지하고, 사진 분석과 보고서 생성은 큐 기반 워커로 분리하며, Revit Desktop과의 상호작용은 Add-in이 맡고, 웹 모델은 APS Viewer가 담당한다. APS 보안 지침은 사용자 개입이 있는 데이터 접근에서 3-legged 인증을 우선하고, 2-legged는 사용자 없는 계정 수준 흐름에 제한해야 한다고 권고한다. 또한 APS Viewer는 서버 프록시를 통해 토큰을 숨기는 구성이 권장된다. [9]

```
flowchart LR
    subgraph Client
        MW[Mobile App]
        WW[Web Admin Console]
        RA[Revit Add-in]
    end

    subgraph Platform
        API[Backend API]
        AUTH[Auth and RBAC]
        QUEUE[Redis and Job Queue]
        AIW[AI Analysis Worker]
        REPW[Report Worker]
        MCP[MCP Server]
```

```
    end

    subgraph Data
        PG[(PostgreSQL)]
        OBJ[(Object Storage)]
    end

    subgraph AutodeskLayer
        APS[APS Viewer and Model Derivative]
        RVT[Local Revit Desktop]
    end

    MW --> API
    WW --> API
    RA --> API
    WW --> APS
    RA --> RVT

    API --> AUTH
    API --> PG
    API --> OBJ
    API --> QUEUE
    QUEUE --> AIW
    QUEUE --> REPW
    AIW --> PG
    REPW --> PG
    AIW --> OBJ
    REPW --> OBJ

    API --> APS
    MCP --> API
    MCP --> RA
```

위 구조에서 API는 JWT 기반 인증과 프로젝트 범위 검증을 수행하고, Storage는 원본 사진·썸네일·보고서 파일을 직접 저장한다. AI 계층은 출력물을 자유 텍스트로 남기지 않고 정형 JSON으로 저장한다. OpenAI [10] OpenAI Responses API는 텍스트·이미지 입력, 함수 호출, 외부 시스템 연결을 지원하고 Structured Outputs는 JSON mode보다 엄격한 schema adherence를 제공한다. 대안으로 Google [11] Google Gemini도 image understanding과 JSON Schema 기반 structured output을 제공하므로 벤더 이중화를 고려할 수 있다. [12]

권고 스택

계층	권고 기술	채택 이유
Web	Next.js, React, TypeScript, Tailwind CSS	관리자 UI, APS 임베딩, 생산성
Mobile	React Native, Expo, TypeScript	현장 앱, 카메라, 오프라인 큐, TS 공유

계층	권고 기술	채택 이유
Backend	NestJS, TypeScript	모듈화, DTO/Validation, WebSocket/SSE
DB	PostgreSQL	강한 관계 모델, JSONB, 인덱싱
Queue	Redis + BullMQ 계열	AI/보고서 비동기 처리
Object Storage	S3-compatible	direct upload, vendor abstraction
AI	OpenAI primary, Gemini secondary	구조화 출력과 멀티모달
BIM Desktop	C#/.NET 8, Revit API, WPF Dockable Pane	Revit 네이티브 연동
Web BIM	APS Viewer, Model Derivative	웹 모델 조회·속성 탐색
Automation	MCP Server	자연어 자동화와 도구 계층

배포와 운영 기준

영역	권고
API 배포	컨테이너 기반 수평 확장
Queue	별도 워커 풀, AI 워커와 보고서 워커 분리
DB	관리형 PostgreSQL, PITR, 읽기 부하 증가 시 replica 고려
Storage	private bucket 기본, signed URL 만료 정책
CDN	썸네일·정적 자산 우선
Observability	중앙 로그, 요청 추적 ID, 큐 적체 모니터링
Security	company_id/project_id 범위 강제, audit log 의무화
APS	rate-limit 대응, backoff, usage 모니터링
Revit	버전별 Add-in 빌드 분리, 2025+ 우선 지원

APS는 API rate limits를 두고 있으며, Autodesk 공식 가이드는 `X-RateLimit-*` 헤더 모니터링과 exponential backoff를 권고한다. 따라서 API 계층에는 APS 호출에 대한 재시도 정책, 큐형 번역 요청, 장애 격리 회로가 필요하다. [13]

데이터 모델과 API

시스템의 모든 업무 단위는 **Project → Level → Room → Photo** 계층 위에 놓는다. Report는 특정 시점의 결과물이므로 Photo와 N:M 관계를 가지는 스냅샷 구조로 보관한다. PostgreSQL 공식 문서가 설명하듯 외래 키는 `NO ACTION`, `RESTRICT`, `CASCADE`, `SET NULL`, `SET DEFAULT`를 제공하고, `CASCADE`는 자식이 부모의 구성요소일 때, `RESTRICT/NO ACTION`은 독립 객체일 때 쓰는 것이 적절하다. JSONB 검색에는 GIN 인덱스가 적합하고, 부분 인덱스는 리뷰 대기 등 일부 데이터 집합을 빠르게 조회하는 데 유리하다. [14]

ERD와 관계 규칙

```
erDiagram
    companies ||--o{ users : has
    companies ||--o{ projects : owns

    users ||--o{ project_members : joins
    projects ||--o{ project_members : grants

    projects ||--o{ revit_models : contains
    revit_models ||--o{ levels : has
    projects ||--o{ levels : maintains

    projects ||--o{ rooms : contains
    levels ||--o{ rooms : groups
    revit_models ||--o{ rooms : syncs

    projects ||--o{ photos : contains
    rooms ||--o{ photos : has
    users ||--o{ photos : uploads

    photos ||--o{ photo_ai_analyses : analyzed_by
    photos ||--o{ photo_comments : has
    users ||--o{ photo_comments : writes

    rooms ||--o{ room_statuses : summarizes

    projects ||--o{ reports : has
    reports ||--o{ report_photos : includes
    photos ||--o{ report_photos : references

    projects ||--o{ audit_logs : records
    users ||--o{ audit_logs : triggers
```

관계·FK·삭제 정책

부모	자식	관계	FK 정책	삭제 정책	이유
companies	users	1:N	`users.company_id`	RESTRICT	회사 삭제 전 사용자 정리 필요

부모	자식	관계	FK 정책	삭제 정책	이유
companies	projects	1:N	`projects.company_id`	RESTRICT	프로젝트 존재 시 회사 hard delete 금지
projects	project_members	1:N	`project_members.project_id`	CASCADE	프로젝트 purge 구성요소
users	project_members	1:N	`project_members.user_id`	CASCADE	사용자 purge 시 멤버십 제거
projects	revit_models	1:N	`revit_models.project_id`	CASCADE	프로젝트 purge 종속
levels	rooms	1:N	`rooms.level_id`	SET NULL	Level 이력 변경 시 Room 보존
revit_models	rooms	1:N	`rooms.revit_model_id`	SET NULL	모델 교체 시 Room 이력 보존
projects	rooms	1:N	`rooms.project_id`	CASCADE	프로젝트 purge 시 제거
rooms	photos	1:N	`photos.room_id`	CASCADE	Room 종속 사진
photos	photo_ai_analyses	1:N	`photo_ai_analyses.photo_id`	CASCADE	사진 종속 분석

부모	자식	관계	FK 정책	삭제 정책	이유
photos	photo_comments	1:N	`photo_comments.photo_id`	CASCADE	사진 종속 코멘트
rooms	room_statuses	1:N	`room_statuses.room_id`	CASCADE	Room 종속 상태
projects	reports	1:N	`reports.project_id`	CASCADE	프로젝트 purge 시 제거
reports	report_photos	1:N	`report_photos.report_id`	CASCADE	보고서 종속
photos	report_photos	1:N	`report_photos.photo_id`	SET NULL	보고서 스냅샷 보존

핵심 인덱스

테이블	인덱스	목적
rooms	`uq_rooms_bim_photo_room_id`	Revit-백엔드 Room 고유 매핑
rooms	`idx_rooms_project_level_name`	층/Room 브라우징
photos	`idx_photos_project_room_workdate_desc`	Room 타임라인
photos	`idx_photos_project_trade_surface_date`	공종·공사면·기간 필터
photos	`idx_photos_uploaded_by_date`	작업자별 추적
photo_ai_analyses	`gin_photo_ai_result_json`	JSONB 검색
photo_ai_analyses	`idx_photo_ai_review_queue` 부분 인덱스	검토 대기 큐
reports	`idx_reports_project_created_desc`	최근 보고서 조회
room_statuses	`idx_room_statuses_project_room_scope`	오버레이 조회

샘플 SQL DDL

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE rooms (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```sql
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    revit_model_id UUID NULL REFERENCES revit_models(id) ON DELETE SET NULL,
    level_id UUID NULL REFERENCES levels(id) ON DELETE SET NULL,
    bim_photo_room_id TEXT NOT NULL UNIQUE,
    revit_unique_id TEXT NULL UNIQUE,
    revit_element_id TEXT NULL,
    room_number TEXT NULL,
    room_name TEXT NOT NULL,
    level_name TEXT NULL,
    area_m2 NUMERIC(12,2) NULL,
    location_text TEXT NULL,
    status TEXT NOT NULL DEFAULT 'ACTIVE'
        CHECK (status IN ('ACTIVE','ARCHIVED','DELETED')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ NULL
);

CREATE INDEX idx_rooms_project_level_name
    ON rooms(project_id, level_name, room_name);

CREATE TABLE photos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    project_id UUID NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
    uploaded_by UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
    work_surface_code TEXT NOT NULL,
    trade_code TEXT NOT NULL,
    work_date DATE NOT NULL,
    worker_name TEXT NULL,
    user_description TEXT NULL,
    ai_description TEXT NULL,
    progress_status TEXT NOT NULL DEFAULT 'PENDING_REVIEW'
        CHECK (progress_status IN
('NOT_STARTED','IN_PROGRESS','COMPLETED','BLOCKED','PENDING_REVIEW')),
    storage_provider TEXT NOT NULL CHECK (storage_provider IN ('S3','R2')),
    object_key TEXT NOT NULL,
    original_url TEXT NULL,
    thumbnail_url TEXT NULL,
    mime_type TEXT NOT NULL,
    file_size BIGINT NOT NULL CHECK (file_size > 0),
    width INT NULL,
    height INT NULL,
    checksum_sha256 TEXT NULL,
    taken_at TIMESTAMPTZ NULL,
    uploaded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    status TEXT NOT NULL DEFAULT 'ACTIVE'
        CHECK (status IN ('ACTIVE','ARCHIVED','DELETED'))
```

```sql
);

CREATE INDEX idx_photos_project_room_workdate_desc
    ON photos(project_id, room_id, work_date DESC, uploaded_at DESC);

CREATE INDEX idx_photos_project_trade_surface_date
    ON photos(project_id, trade_code, work_surface_code, work_date DESC);

CREATE TABLE photo_ai_analyses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    photo_id UUID NOT NULL REFERENCES photos(id) ON DELETE CASCADE,
    model_provider TEXT NOT NULL CHECK (model_provider IN ('OPENAI','GEMINI')),
    model_name TEXT NOT NULL,
    prompt_version TEXT NOT NULL,
    result_json JSONB NOT NULL,
    summary TEXT NOT NULL,
    detected_trade_code TEXT NULL,
    detected_surface_code TEXT NULL,
    progress_status TEXT NOT NULL
        CHECK (progress_status IN
('NOT_STARTED','IN_PROGRESS','COMPLETED','BLOCKED','PENDING_REVIEW')),
    confidence NUMERIC(5,4) NOT NULL CHECK (confidence >= 0 AND confidence <=
1),
    requires_human_review BOOLEAN NOT NULL DEFAULT TRUE,
    reviewed_by UUID NULL REFERENCES users(id) ON DELETE SET NULL,
    reviewed_at TIMESTAMPTZ NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_photo_ai_review_queue
    ON photo_ai_analyses(created_at DESC)
    WHERE requires_human_review = TRUE;

CREATE INDEX gin_photo_ai_result_json
    ON photo_ai_analyses USING GIN (result_json);
```

이 DDL은 PostgreSQL의 FK delete actions, JSONB GIN indexing, partial index 설계를 그대로 활용한 기준안이다. 실제 운영에서는 대부분 soft delete를 우선 적용하고, hard delete는 purge job에서만 수행한다. [14]

API 공통 규약

항목	규약
Base URL	`/api/v1`
인증	`Authorization: Bearer <jwt>`

항목	규약
멀티테넌시	`company_id`, `project_id` 범위 검증 필수
시간	저장 UTC ISO 8601, 표시만 로컬
페이지네이션	`page`, `page_size`, `total`, `has_next`
정렬	`sort`, `order`
멱등성	업로드 완료 커밋, 보고서 생성에 `Idempotency-Key` 지원
에러 포맷	`error.code`, `message`, `details`, `request_id`

권한 모델

역할	API 범위
Worker	사진 업로드, 본인/프로젝트 조회
Manager	AI 검토, 상태 변경, 빠른 보고서
Project Admin	프로젝트 설정, 보고서, 키 관리
BIM Manager	Revit sync, 상태 오버레이, 매핑 관리
Company Admin	회사/사용자/전체 프로젝트 운영
Viewer	조회만 가능

엔드포인트 카탈로그

도메인	메서드	경로
Auth	`POST`	`/auth/register`
Auth	`POST`	`/auth/login`
Auth	`POST`	`/auth/refresh`
Auth	`GET`	`/auth/me`
Projects	`GET`	`/projects`
Projects	`POST`	`/projects`
Projects	`GET`	`/projects/{project_id}`
Rooms	`GET`	`/projects/{project_id}/rooms`
Rooms	`PATCH`	`/rooms/{room_id}`
Uploads	`POST`	`/uploads/photos/presign`

도메인	메서드	경로
Photos	`POST`	`/photos`
Photos	`GET`	`/photos`
Photos	`GET`	`/photos/{photo_id}`
Photos	`PATCH`	`/photos/{photo_id}`
Analysis	`GET`	`/photos/{photo_id}/analysis`
Analysis	`PATCH`	`/photos/{photo_id}/analysis/review`
Reports	`POST`	`/reports`
Reports	`GET`	`/reports`
Reports	`GET`	`/reports/{report_id}`
Reports	`POST`	`/reports/{report_id}/generate`
Revit	`POST`	`/revit/connect`
Revit	`POST`	`/revit/sync-rooms`
Revit	`GET`	`/revit/rooms/{bim_photo_room_id}/photos`
Revit	`GET`	`/revit/projects/{project_id}/room-statuses`
APS	`POST`	`/aps/models`
APS	`GET`	`/aps/models/{model_id}/status`
APS	`GET`	`/aps/models/{model_id}/viewer-token`
Admin	`GET`	`/admin/dashboard`
Admin	`GET`	`/admin/jobs`
Admin	`GET`	`/admin/audit-logs`

상세 엔드포인트 명세

`POST /api/v1/photos`

목적
presigned URL 업로드가 완료된 사진의 메타데이터를 저장하고 AI 분석 작업을 큐에 등록한다.

권한
`worker`, `manager`, `project_admin`, `company_admin`

검증 논리
`project_id`는 로그인 사용자가 접근 가능한 프로젝트여야 하며, `room_id`는 해당 프로젝트 소속이어야 한다.

`upload_id`는 presign 단계에서 발급되었고 만료되지 않았어야 한다. `work_date`는 프로젝트 정책상 허용 범위를 벗어나면 거부한다. `user_description`은 sanitizer를 거쳐 저장한다. 파일의 mime, size, checksum은 presign 메타와 일치해야 한다. S3 presigned URL과 R2 direct upload 모두 time-limited direct upload에 적합하므로, API 서버는 대형 바이너리를 프록시하지 않고 메타데이터 커밋만 처리하는 구조를 사용한다. [7]

요청 JSON Schema

```
{
  "type": "object",
  "additionalProperties": false,
  "required": [
    "project_id",
    "room_id",
    "upload_id",
    "work_surface_code",
    "trade_code",
    "work_date"
  ],
  "properties": {
    "project_id": { "type": "string", "format": "uuid" },
    "room_id": { "type": "string", "format": "uuid" },
    "upload_id": { "type": "string", "format": "uuid" },
    "work_surface_code": {
      "type": "string",
      "enum": ["FLOOR", "WALL", "CEILING", "WINDOW", "DOOR", "PIPE",
"ELECTRIC", "OTHER"]
    },
    "trade_code": {
      "type": "string",
      "enum": ["WATERPROOF", "TILE", "PAINT", "ELECTRIC", "MEP", "WINDOW",
"CONCRETE", "OTHER"]
    },
    "work_date": { "type": "string", "format": "date" },
    "worker_name": { "type": ["string", "null"], "maxLength": 100 },
    "user_description": { "type": ["string", "null"], "maxLength": 5000 },
    "taken_at": { "type": ["string", "null"], "format": "date-time" },
    "gps": {
      "type": ["object", "null"],
      "additionalProperties": false,
      "properties": {
        "lat": { "type": "number", "minimum": -90, "maximum": 90 },
        "lng": { "type": "number", "minimum": -180, "maximum": 180 }
      }
    }
```

```
  }
}
```

예시 요청

```
POST /api/v1/photos HTTP/1.1
Authorization: Bearer eyJ...
Content-Type: application/json
Idempotency-Key: 4aa2d7f0-31ad-4776-84b3-fbbc3d9d7d78

{
  "project_id": "c949b081-d1a9-4db5-b97d-b66d6bfd7b90",
  "room_id": "f0c6f393-f5b5-46cd-8e43-9d4aa777fbb0",
  "upload_id": "ebbd0a67-f2b6-44aa-b8d7-d9d7b3f4d1d1",
  "work_surface_code": "FLOOR",
  "trade_code": "WATERPROOF",
  "work_date": "2026-03-05",
  "worker_name": "김작업",
  "user_description": "배수구 주변 2차 방수 진행",
  "taken_at": "2026-03-05T09:12:44+09:00"
}
```

성공 응답

```
{
  "data": {
    "photo_id": "0a907e31-f0db-4f98-8a2e-f6c6c4a6ef9d",
    "project_id": "c949b081-d1a9-4db5-b97d-b66d6bfd7b90",
    "room_id": "f0c6f393-f5b5-46cd-8e43-9d4aa777fbb0",
    "status": "ACTIVE",
    "progress_status": "PENDING_REVIEW",
    "analysis_job": {
      "job_id": "job_01JZ...",
      "status": "QUEUED"
    },
    "storage": {
      "provider": "S3",
      "object_key": "photos/c949.../2026/03/05/uuid.jpg",
      "thumbnail_url": null
    },
    "created_at": "2026-04-28T05:15:22Z"
  }
}
```

오류 응답

```
{
  "error": {
    "code": "ROOM_NOT_IN_PROJECT",
    "message": "room_id is not part of the specified project.",
    "details": {
      "project_id": "c949b081-d1a9-4db5-b97d-b66d6bfd7b90",
      "room_id": "f0c6f393-f5b5-46cd-8e43-9d4aa777fbb0"
    },
    "request_id": "req_01JZ8P..."
  }
}
```

표준 에러 코드

코드	HTTP	의미
`AUTH_REQUIRED`	401	인증 토큰 부재/만료
`FORBIDDEN_ROLE`	403	역할 부족
`PROJECT_SCOPE_MISMATCH`	403	다른 프로젝트 접근
`VALIDATION_FAILED`	422	필드 형식 오류
`ROOM_NOT_FOUND`	404	Room 없음
`ROOM_NOT_IN_PROJECT`	409	Room-Project 불일치
`UPLOAD_NOT_FOUND`	404	presign 업로드 없음
`REVIT_SYNC_CONFLICT`	409	Room 식별자 충돌
`AI_PROVIDER_TIMEOUT`	504	AI 분석 시간 초과
`REPORT_ALREADY_RUNNING`	409	동일 조건 보고서 생성 중
`APS_TRANSLATION_PENDING`	409	APS 변환 미완료

핵심 흐름과 Revit 연동

사진 업로드, Room 동기화, Room 클릭 조회, 보고서 생성은 모두 비동기 이벤트와 명시적 데이터 경계를 갖도록 설계한다. 특히 모바일 앱과 Revit Add-in은 각각 현장 입력과 BIM 조회에 집중해야 하며, 장시간 작업은 큐와 워커가 맡는다.

사진 업로드와 AI 분석

```
sequenceDiagram
    autonumber
    participant U as User
```

```
    participant API as Backend API
    participant OBJ as Object Storage
    participant DB as PostgreSQL
    participant Q as Queue
    participant AI as AI Worker
    participant LLM as Vision LLM

    U->>API: POST /uploads/photos/presign
    API-->>U: upload_id + presigned_url + object_key
    U->>OBJ: PUT file(binary)
    OBJ-->>U: 200 OK
    U->>API: POST /photos(metadata)
    API->>DB: insert photo row
    API->>Q: enqueue analyze_photo(photo_id)
    API-->>U: 201 Created
    Q->>AI: consume job
    AI->>LLM: image + metadata + JSON schema
    LLM-->>AI: structured analysis JSON
    AI->>DB: insert analysis + update photo summary
```

이 설계는 S3/R2 direct upload와 구조화된 AI 결과 저장을 전제로 한다. S3 presigned URL은 time-limited PUT/GET/HEAD를 지원하고, OpenAI Responses/Structured Outputs, Gemini structured output은 JSON schema 기반 결과를 저장하기에 적합하다. [15]

Revit Room 동기화

```
sequenceDiagram
    autonumber
    participant BIM as BIM Manager
    participant ADDIN as Revit Add-in
    participant RVT as Revit Model
    participant API as Backend API
    participant DB as PostgreSQL

    BIM->>ADDIN: 프로젝트 연결 / Room 동기화 실행
    ADDIN->>RVT: Room 스캔
    ADDIN->>API: POST /revit/sync-rooms(rooms[])
    API->>DB: upsert by revit_unique_id / bim_photo_room_id
    DB-->>API: room mappings
    API-->>ADDIN: room_id + bim_photo_room_id
    ADDIN->>RVT: BIM_PHOTO_ROOM_ID write-back
    ADDIN-->>BIM: 신규/수정/충돌 요약
```

Revit 모델 내 데이터 저장은 shared parameter 또는 extensible storage가 가능하지만, Autodesk 문서가 보여주듯 shared parameter는 생성 시 사용자에게 숨김 처리(`Visible=false`)할 수 있고, 정의 파일 기반으로 배포와 검증이 용이하다. 본 시스템은 운영성과 가시성을 위해 `BIM_PHOTO_ROOM_ID`를 shared parameter로 채택한다. [3]

Revit Room 클릭과 사진 조회

```
sequenceDiagram
    autonumber
    participant U as BIM User
    participant ADDIN as Revit Add-in
    participant RVT as Revit
    participant API as Backend API
    participant DB as PostgreSQL
    participant OBJ as Object Storage

    U->>RVT: Room 선택
    ADDIN->>RVT: 선택 요소 확인
    ADDIN->>ADDIN: BIM_PHOTO_ROOM_ID 읽기
    ADDIN->>API: GET /revit/rooms/{bim_photo_room_id}/photos
    API->>DB: query photos + latest analyses
    API->>OBJ: sign thumbnail urls
    API-->>ADDIN: room summary + photo timeline
    ADDIN-->>U: Dockable Panel 갱신
```

Revit의 modeless UI는 External Events 프레임워크를 사용하는 것이 공식 권장 방식이며, Dockable Pane은 Revit 창 좌우상하 및 기존 시스템 패널 탭으로 도킹할 수 있다. 따라서 패널 UI는 modeless로 유지하되, 모델 쓰기 작업은 External Event를 통해 Revit 메인 스레드에서 처리해야 한다. [16]

보고서 생성

```
sequenceDiagram
    autonumber
    participant M as Manager
    participant API as Backend API
    participant DB as PostgreSQL
    participant Q as Queue
    participant RW as Report Worker
    participant LLM as LLM
    participant OBJ as Object Storage

    M->>API: POST /reports {filters, formats}
    API->>DB: create report(status=QUEUED)
    API->>Q: enqueue generate_report(report_id)
    API-->>M: 202 Accepted
    Q->>RW: consume job
    RW->>DB: load photos + analyses + room metadata
    RW->>LLM: compare timeline and generate summary JSON
    LLM-->>RW: report JSON
```

```
    RW->>OBJ: write PDF/DOCX/HWP(if enabled)
    RW->>DB: update report(status=READY, file_urls)
```

Revit Add-in 명세

Revit Add-in은 `.addin` manifest를 통해 로드되는 `IExternalApplication` 기반 외부 애플리케이션이며, `OnStartup`과 `OnShutdown` 생명주기를 갖는다. 리본 탭과 커스텀 패널은 `UIControlledApplication`에서 등록한다. Revit 2025 공식 개발 요구사항은 .NET 8.0과 `RevitAPI.dll`, `RevitAPIUI.dll` 참조를 요구한다. [17]

명령과 이벤트

구성	종류	역할
`BimPhotoSyncApp`	`IExternalApplication`	리본, Dockable Pane, 이벤트 등록
`ConnectProjectCommand`	`IExternalCommand`	로그인·프로젝트 연결
`SyncRoomsCommand`	`IExternalCommand` 또는 `ExternalEvent`	Room sync 및 write-back
`PhotoPaneProvider`	Dockable Pane	Room 사진 패널 UI
`SelectionRefresh`	선택 변화 감시	선택 Room의 패널 갱신
`ApplyOverlayCommand`	`ExternalEvent`	상태 오버레이 적용
`ClearOverlayCommand`	`ExternalEvent`	오버레이 초기화

Dockable Panel 목업

```
+-----------------------------------------------------------------+
| BIM Photo Sync                                                  |
| Project: A현장 / Model: ARCH_2025_04                          |
| Room: 101 욕실   Level: 3F   BIM_PHOTO_ROOM_ID: rm_01JZ...      |
|-----------------------------------------------------------------|
| Status: 진행중   [오버레이 적용] [새로고침] [웹에서 열기] [보고서]|
| Filters: 기간 [7일▼]  공종 [방수▼]  면 [바닥▼]                |
|-----------------------------------------------------------------|
| 2026-03-05 09:12  방수공사 / 김작업                           |
| [썸네일] AI: 배수구 주변 추가 확인 필요                          |
|-----------------------------------------------------------------|
| 2026-03-04 14:20  방수공사 / 김작업                           |
| [썸네일] AI: 모서리 보강 확인됨                                  |
|-----------------------------------------------------------------|
| Latest Summary                                                  |
| - 방수 2차 진행                                                 |
| - 완료율 추정 65%                                               |
```

```
| - 관리자 검토 필요                                          |
+-------------------------------------------------------------+
```

공정 색상 표시 원칙

Room 상태는 뷰 단위 오버레이로 관리한다. Revit API의 View Filters 문서는 `View.SetFilterOverrides()`와 `View.SetFilterVisibility()`로 뷰의 그래픽 오버라이드를 적용할 수 있음을 보여준다. 따라서 본 기획안은 Room별 상태 맵을 필터 또는 선택 필터 계층으로 변환해 현재 뷰에 오버레이를 적용하는 방식을 권장한다. [18]

상태	색상	의미
`NOT_STARTED`	빨강	시작 전
`IN_PROGRESS`	주황	진행 중
`COMPLETED`	초록	완료
`BLOCKED`	보라	이슈/차단
`PENDING_REVIEW`	회색	검토 대기

Revit Add-in 의사코드

```
public class BimPhotoSyncApp : IExternalApplication
{
    public Result OnStartup(UIControlledApplication app)
    {
        RibbonBuilder.CreateTabAndButtons(app);
        DockablePaneRegistry.RegisterPane(app, new PhotoPaneProvider());
        EventBus.RegisterSelectionRefresh(app);
        return Result.Succeeded;
    }

    public Result OnShutdown(UIControlledApplication app)
    {
        EventBus.Unregister(app);
        return Result.Succeeded;
    }
}

public class SyncRoomsExternalHandler : IExternalEventHandler
{
    public void Execute(UIApplication uiapp)
    {
        Document doc = uiapp.ActiveUIDocument.Document;

        var rooms = new FilteredElementCollector(doc)
```

```
            .OfCategory(BuiltInCategory.OST_Rooms)
            .WhereElementIsNotElementType()
            .Cast<SpatialElement>()
            .Select(r => new SyncRoomDto
            {
                RevitUniqueId = r.UniqueId,
                RevitElementId = r.Id.IntegerValue.ToString(),
                RoomNumber =
r.get_Parameter(BuiltInParameter.ROOM_NUMBER)?.AsString(),
                RoomName =
r.get_Parameter(BuiltInParameter.ROOM_NAME)?.AsString(),
                LevelName = doc.GetElement(r.LevelId)?.Name
            })
            .ToList();

        var response = ApiClient.SyncRooms(rooms);

        using (Transaction tx = new Transaction(doc, "Write BIM_PHOTO_ROOM_ID"))
        {
            tx.Start();

            foreach (var mapped in response.RoomMappings)
            {
                var element = doc.GetElement(new
ElementId(int.Parse(mapped.RevitElementId)));
                var param = element?.LookupParameter("BIM_PHOTO_ROOM_ID");
                if (param != null && !param.IsReadOnly)
                {
                    param.Set(mapped.BimPhotoRoomId);
                }
            }

            tx.Commit();
        }
    }

    public string GetName() => "BIM Photo Sync Room Sync";
}
```

이 의사코드는 `IExternalApplication` 기반 시작점, Room collector, 서버 sync, shared parameter write-back, Transaction 기반 모델 수정이라는 운영 기준을 보여준다. Autodesk 문서 기준으로 Add-in은 `.addin` manifest, `IExternalApplication`, Dockable Pane, External Events 조합으로 구현해야 한다. [19]

호환성과 전제

항목	기준
지원 Revit	1차: 2025+, 2차: 2024 별도 검토
런타임	.NET 8.0
저장 방식	`BIM_PHOTO_ROOM_ID` shared parameter
패널	WPF Dockable Pane
모델 쓰기	External Event에서만 수행
오프라인	제한적 캐시만 허용, 서버 기준 우선
Autodesk MCP	참고 대상으로만 사용, 핵심 의존성 아님

Autodesk는 2026년 공개 문서에서 Autodesk MCP Servers와 Revit MCP Server 방향을 제시하고 있지만, Revit Public MCP Server는 2027 Tech Preview 단계다. 따라서 본 시스템에서 MCP는 자체 서버와 Add-in 브리지를 우선 설계하고, Autodesk의 공용 MCP는 장기 옵션으로 둔다. [20]

웹·앱 UX와 MCP 자동화

웹은 정밀 관리·비교·보고서·Viewer에 집중하고, 모바일은 촬영·업로드·승인·빠른 보고서에 집중한다. 관리자도 앱을 쓸 수 있지만, 권한 관리·Room 충돌 해결·멀티패널 Viewer 같은 고밀도 작업은 웹 전용으로 두는 것이 맞다. 이 분리는 현재 요구사항의 "현장 이동 중 관리자 사용"과 "정밀 BIM·보고서·권한 관리"를 동시에 충족하기 위한 핵심 UX 원칙이다.

사용자 여정

역할	시작점	주요 흐름	종료 상태
Worker	모바일 홈	촬영 → Room 선택 → 메타데이터 입력 → 업로드	사진이 Room에 연결되고 AI 큐 등록
Manager	웹/앱 대시보드	AI 검토 → 상태 변경 → 빠른 보고서 또는 상세 보고서	상태 확정 및 보고서 생성
BIM Manager	Revit 또는 웹 설정	프로젝트 연결 → Room sync → Room 클릭 조회 → 상태 오버레이	BIM-현장 데이터 연결 완료

웹 화면 목록

화면	목적
로그인 / 회사 참여	인증과 프로젝트 접근
프로젝트 선택	접근 가능한 현장 진입
대시보드	상태 요약, 검토 대기, 업로드 현황

화면	목적
Room 관리	층·Room 브라우징, 상태 요약
사진 관리	고급 필터·비교·AI 검토
사진 상세	메타데이터, 분석 결과, 승인/수정
보고서 빌더	조건 조합, 제목, 포맷, 생성
보고서 상세	미리보기, 파일 다운로드, 재생성
APS Viewer	모델 탐색, Room 선택, 패널 표시
Revit Sync 관리	동기화 상태, 충돌, Add-in 연결
사용자/권한 관리	역할, 접근키, 관리자키
감사/Jobs	로그, 실패 작업, 재시도

모바일 화면 목록

화면	목적
로그인	인증
프로젝트 선택	현장 선택
작업자 홈	촬영/앨범/대기열 진입
사진 촬영	연속 촬영
메타데이터 입력	Room/공종/공사면/날짜
Room 선택	검색형 선택
업로드 대기열	오프라인/실패 재시도
내 업로드	작업자 이력
관리자 대시보드	오늘 업로드, 검토 대기, Room 상태
AI 검토	승인/수정/재분석
Room 현황	Room별 상태와 타임라인
빠른 보고서	Room+기간 PDF 생성
작업자 현황	담당자별 업로드 현황

UX reference board

위 시각 참고 이미지는 “산업형 관리자 대시보드”, “현장 사진 업로드 앱”, “웹 BIM 뷰어”, “데스크톱 사이드 패널”의 무드보드를 만들기 위한 참고 방향이다. 실제 구현 시에는 밝은-어두운 색상보다 상태 색상 대비, 큰 썸네일, Room/공종 검색성, 터치 목표 크기, 승인 버튼의 위치, 작업 큐 시각화를 우선해야 한다.

핵심 컴포넌트

채널	컴포넌트	설명
공통	ProjectSwitcher	회사/프로젝트 문맥 전환
공통	RoomPicker	검색 가능한 Room 선택기
공통	TradeSurfaceSelector	공종/공사면 표준 분류 선택
공통	PhotoCard	썸네일·상태·작성자·일자 표시
공통	AnalysisBadge	신뢰도, 검토 필요 표시
Web	PhotoExplorer	그리드/리스트 전환 필터 뷰
Web	TimelineCompare	전·중·후 사진 비교
Web	ReportFilterBuilder	보고서 필터 조합 생성
Web	ViewerSidePanel	APS Room 연동 패널
Mobile	CaptureSheet	카메라·앨범·임시저장
Mobile	UploadQueueList	오프라인 대기열
Mobile	ManagerApprovalSheet	AI 승인/수정 모달
Revit	PhotoDockPane	선택 Room 사진 패널

웹 와이어프레임

```
┌──────────────────────────────────────────────────────────────────────────────────┐
│ Top Nav: Company | Project | Viewer | Reports | Audit | User                      │
├──────────────────┬──────────────────────────────────────────────────────┬──────────┤
│ Left Sidebar     │ Main Content                                         │ Right Panel │
│ - Dashboard      │ [Breadcrumb] Project > 3F > 101 욕실            │ Room Status │
│ - Rooms          │ ┌──────────────────────────────────────────────┐ │ - 진행중    │
│ - Photos         │ │ Filters: Room | Trade | Surface | Date       │ │ - 완료율    │
│ - Reports        │ └──────────────────────────────────────────────┘ │ - 보고서    │
│ - Viewer         │ ┌──────────────────┬──────────────────────────┐ │             │
│ - Settings       │ │ Photo Grid       │ Selected Photo Details   │ │             │
│                  │ │ [thumb][thumb]   │ - metadata               │ │             │
│                  │ │ [thumb][thumb]   │ - AI summary             │ │             │
```

```
│                │ └──────────────┴──────────────────────┘ │              │
└────────────────┴──────────────────────────────────────────┴──────────────┘
```

모바일 와이어프레임

```
┌──────────────────────┐
│ A현장 ▼      09:41   │
├──────────────────────┤
│ [빠른 촬영] [앨범]    │
│----------------------│
│ Room      [101 욕실▼] │
│ Surface   [바닥▼]     │
│ Trade     [방수▼]     │
│ Work Date [2026-03-05]│
│ Worker    [김작업]    │
│----------------------│
│ 메모                  │
│ 배수구 주변 2차 방수 진행 │
│----------------------│
│ [사진 미리보기]        │
│ [사진 미리보기]        │
│----------------------│
│ [임시저장] [업로드]    │
└──────────────────────┘
```

관리자 모바일 와이어프레임

```
┌──────────────────────┐
│ A현장 관리자 대시보드   │
├──────────────────────┤
│ 오늘 업로드 42장       │
│ AI 검토 대기 8건       │
│ 완료 Room 17개         │
│ 진행중 Room 45개       │
│ 시작전 Room 24개       │
│                       │
│ [AI 검토 대기]          │
│ [Room 현황]             │
│ [빠른 보고서]          │
│ [작업자 현황]          │
└──────────────────────┘
```

MCP 도구 명세

MCP는 JSON-RPC 2.0 기반 표준 프로토콜이며, lifecycle은 initialize → operation → shutdown, HTTP 기반 transport에서는 authorization flow를 정의한다. MCP tools는 모델이 외부 시스템에 대해 검색, API 호출, 계산 등을 수행하도록 하는 서버 기능이고, 명세는 보안과 신뢰를 위해 human-in-the-loop를 권고한다. OpenAI 공식 문서도 Responses API에서 remote MCP server를 `mcp` built-in tool로 연결할 수 있고, 필요 시 OAuth authorization을 함께 전달한다고 설명한다. [21]

도구	입력 스키마 요약	출력 스키마 요약	부작용	승인
`project.search`	`{query, company_scope}`	`{projects[]}`	없음	자동
`room.get`	`{project_id, room_id? , bim_photo_room_id?}`	`{room}`	없음	자동
`photo.search`	`{project_id, room_id?, trade_code?, date_from?, date_to?}`	`{photos[]}`	없음	자동
`photo.analyze_batch`	`{photo_ids[], prompt_profile}`	`{summary, issues, confidence}`	분석 결과 저장 가능	관리자 승인
`report.create`	`{project_id, filters, format}`	`{report_id, status}`	보고서 작업 생성	관리자 승인
`revit.sync_rooms`	`{project_id, revit_context_ref}`	`{created, updated, conflicts}`	DB/모델 수정	BIM 승인
`revit.get_selected_room`	`{}`	`{room_id, bim_photo_room_id}`	없음	자동
`revit.apply_status_overlay`	`{project_id, scope, color_profile}`	`{applied_count}`	Revit 뷰 수정	BIM 승인

MCP 워크플로

```
flowchart TD
    U[관리자 명령] --> A[Agent]
    A --> T1[photo.search]
    T1 --> DB[(Project DB)]
    DB --> T1
    T1 --> A
    A --> T2{분석 또는 보고서 생성?}
    T2 -->|분석| T3[photo.analyze_batch]
    T2 -->|보고서| T4[report.create]
    T3 --> AI[AI Worker]
    T4 --> RW[Report Worker]
    AI --> DB
    RW --> DB
    RW --> OBJ[(Object Storage)]
    A --> H{쓰기 작업 승인}
    H -->|승인| DONE[링크와 요약 반환]
    H -->|거부| SAFE[읽기 결과만 반환]
```

MCP 보안 제약

원칙	정책
전송	HTTPS 또는 로컬 stdio/named pipe만 허용
초기화	`initialize` 완료 전 임의 호출 금지
범위	company/project scope를 모든 호출에 포함
승인	읽기 자동, 쓰기 승인 필수
감사	actor, tool, args hash, result hash 기록
세션	장기 토큰은 서버만 보관, 클라이언트는 단기 세션 사용
실패	재시도 가능 작업과 비가역 작업 분리

검증·전환·운영 계획

품질 관리는 단순 단위 테스트로 끝나면 안 된다. 이 시스템은 모바일 업로드, 객체 스토리지, AI 워커, 보고서 렌더러, APS, Revit Add-in, MCP까지 걸쳐 있으므로 계약 테스트와 통합 테스트가 필수다. 또한 기존 앱은 “운영 제품”이 아니라 “자산 추출 원본”으로만 취급해야 한다.

테스트와 QA

테스트 층	대상	기준
Unit	검증 함수, 상태 판정, 제목 생성기	높은 커버리지와 deterministic rules
Contract	OpenAPI, JSON Schema, MCP tool schema	breaking change 0
Integration	presign-upload-commit, AI/Report worker	저장소·큐·DB 실연결 통과
E2E Web	관리자 주요 시나리오	업로드 조회/필터/보고서 통과
E2E Mobile	촬영·오프라인 큐·재시도	불안정 네트워크에서도 성공
E2E Revit	Room sync, write-back, panel, overlay	대상 Revit 버전 통과
Security	권한 상승, 범위 우회, signed URL 오남용	위협 시나리오 차단
Performance	대량 사진 목록, 동시 보고서 생성	SLA 충족

전환 계획

기존 앱은 코드 재사용 대상이 아니다. 이관은 “파일·메타데이터·Room명 텍스트”를 새 모델로 옮기는 데이터 프로젝트다. 따라서 컷오버 전에 다음 순서를 따른다.

단계	내용
인벤토리	기존 스토리지와 메타데이터 목록화
추출	원본 사진, 업로드 날짜, 프로젝트명, 방 이름, 작성자 추출
정규화	공종/공사면/날짜/작업자 기준값 통합
Room 매핑	Revit Room 목록과 과거 사진 텍스트 매칭, 충돌 수동 해결
병행 운영	일정 기간 조회만 이중 운영
컷오버	신규 시스템만 쓰고 기존은 읽기 아카이브로 전환

단계와 마일스톤

단계	핵심 산출물	완료 기준
Foundation	Auth, Company/Project, Room 모델, 기준값	프로젝트/Room CRUD 안정화
Field Capture	모바일 업로드, presign, 사진 목록	모바일→웹 동기화 성공
AI and Review	구조화 분석, 승인 화면, 상태 모델	검토/승인/상태 저장 가능
Reporting	PDF/DOCX 생성, 비교 분석	필터 기반 보고서 생성 성공
Revit	Add-in, Room sync, panel, overlay	Room 클릭·오버레이 성공

단계	핵심 산출물	완료 기준
Viewer and MCP	APS Viewer, MCP tools, human approval	Room 컨텍스트 자동화 가능
Rollout	UAT, 운영 문서, 모니터링, 컷오버	운영 승인 완료

납품물

유형	납품물
제품	웹 콘솔, 모바일 앱, Revit Add-in, API, 워커
설계	통합 기획서, ERD, OpenAPI 명세, MCP 명세, Revit 명세
운영	배포 문서, 관리자 매뉴얼, BIM 운영 매뉴얼, 장애 대응 절차
품질	테스트 시나리오, UAT 결과, 보안 점검표, 성능 리포트

인수 기준

항목	기준
Room 귀속	모든 사진은 프로젝트와 Room에 귀속되어야 한다
동기화	모바일 업로드가 웹과 Revit 조회로 이어져야 한다
AI	구조화 JSON 분석 저장과 관리자 수정이 가능해야 한다
보고서	Room/공종/기간 조합으로 PDF/DOCX 생성이 가능해야 한다
Revit	Room sync, `BIM_PHOTO_ROOM_ID` write-back, Room 클릭 조회가 가능해야 한다
오버레이	상태 색상 적용/해제가 가능해야 한다
멀티테넌시	회사 간 데이터 접근이 불가능해야 한다
감사	업로드, 수정, Sync, 보고서, MCP 호출 로그가 남아야 한다

Go-live 체크리스트

체크 항목	완료 기준
인증	SSO 또는 JWT 운영 설정 완료
권한	역할별 API 정책 점검 완료
스토리지	private bucket, signed URL 만료 정책 검증 완료
DB	백업, PITR, 마이그레이션 리허설 완료
AI	프롬프트 버전 고정, fallback 정책 준비
큐	실패 재시도, DLQ 또는 실패 보관 정책 완료

체크 항목	완료 기준
APS	3-legged/프록시 정책, rate limit 모니터링 완료
Revit	대상 버전 설치/배포 절차와 rollback 패키지 준비
모바일	오프라인 큐와 실패 재시도 테스트 완료
웹	관리자 핵심 시나리오 UAT 통과
감사	audit log 조회 화면과 보관 정책 확인
운영	알림, 대시보드, 장애 연락 체계 준비
전환	기존 데이터 추출·정규화·매핑 검수 완료

최종적으로 BIM Photo Sync는 “사진을 올리고 정리하는 서비스”가 아니라, **Room을 기준으로 현장 사진, 상태, 분석, 보고서, Revit, 웹 Viewer, Agent 자동화를 하나의 데이터 체계로 묶는 운영 플랫폼**으로 구축해야 한다. Autodesk의 APS와 Revit API, OpenAI 또는 Google의 구조화된 멀티모달 모델, S3-compatible storage, MCP 표준을 조합하되, 각 계층의 책임을 분리하고 승인·감사·테넌시 경계를 명확히 해야 회사급 품질과 운영 가능성이 나온다. [22]

---

[1] [2] [4] [22] Viewer SDK | Autodesk Platform Services (APS)
https://aps.autodesk.com/developer/overview/viewer-sdk?utm_source=chatgpt.com

[3] Storing Data in the Revit model
https://help.autodesk.com/view/RVT/2024/ENU/?guid=Revit_API_Revit_API_Developers_Guide_Advanced_Topics_Storing_Data_in_the_Revit_model_html&utm_source=chatgpt.com

[5] [7] [15] Download and upload objects with presigned URLs - Amazon Simple Storage Service
https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html

[6] [16] Dockable Dialog Panes
https://help.autodesk.com/cloudhelp/2018/ENU/Revit-API/Revit_API_Developers_Guide/Advanced_Topics/Dockable_Dialog_Panes.html

[8] Get Started with React Native · React Native
https://reactnative.dev/docs/environment-setup

[9] [10] Security Recommendations for BIM 360 App Developers | Autodesk Platform Services
https://aps.autodesk.com/blog/security-recommendations-bim-360-app-developers

[11] [18] https://help.autodesk.com/cloudhelp/2018/ENU/Revit-API/Revit_API_Developers_Guide/Basic_Interaction_with_Revit_Elements/Views/View_Filters.html
https://help.autodesk.com/cloudhelp/2018/ENU/Revit-API/Revit_API_Developers_Guide/Basic_Interaction_with_Revit_Elements/Views/View_Filters.html

[12] Responses Overview | OpenAI API Reference
https://developers.openai.com/api/reference/responses/overview/

[13] https://aps.autodesk.com/en/docs/model-derivative/v2/developers_guide/rate-limiting/md-rate-limits
https://aps.autodesk.com/en/docs/model-derivative/v2/developers_guide/rate-limiting/md-rate-limits

14 https://www.postgresql.org/docs/current/ddl-constraints.html
https://www.postgresql.org/docs/current/ddl-constraints.html

17 19 https://help.autodesk.com/cloudhelp/2018/ENU/Revit-API/Revit_API_Developers_Guide/Introduction/Getting_Started/Using_the_Autodesk_Revit_API/External_Applications.html
https://help.autodesk.com/cloudhelp/2018/ENU/Revit-API/Revit_API_Developers_Guide/Introduction/Getting_Started/Using_the_Autodesk_Revit_API/External_Applications.html

20 https://www.autodesk.com/solutions/autodesk-ai/autodesk-mcp-servers
https://www.autodesk.com/solutions/autodesk-ai/autodesk-mcp-servers

21 Specification - Model Context Protocol
https://modelcontextprotocol.io/specification/2025-03-26